

UML Relationships

Overview

In UML class diagrams, relationships define how classes interact and are structured. The four primary relationships are:

- **Dependency:** A "uses" relationship.
- **Association:** A structural relationship between peer classes.
- **Generalization:** An "is-a" relationship (Inheritance).
- **Realization:** An implementation relationship (Interfaces).

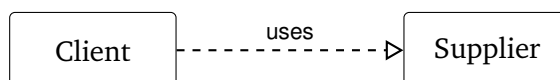
1. Dependency

Dependency

A semantic relationship where a change to one thing (the independent thing) may affect the semantics of the other thing (the dependent thing).

Notation

A **dashed line** with an open arrow pointing to the independent class.



Code Perspective

Typically exists when a class uses another class as a parameter in a method, a local variable, or a return type, but **not** as a field.

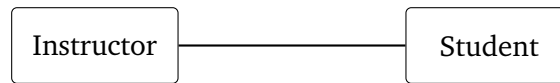
2. Association

Association

A structural relationship that describes a set of links, where a link is a connection among objects. If two classes in a model need to communicate with each other, there must be a link between them.

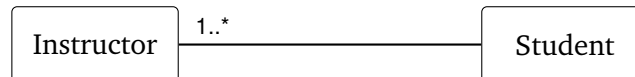
Basic Association

An association denotes a link between classes.



Multiplicity

We can indicate the multiplicity of an association by adding multiplicity adornments to the line.



The example above implies:

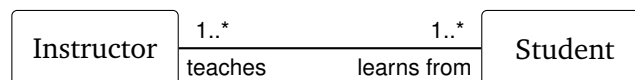
- 1..*: A Student has one or more Instructors.
- *: An Instructor has zero or more Students.

Multiplicity Notation

- 1: Exactly one.
- *: Zero or more.
- 0..1: Zero or one.
- 2..4: Specified range (inclusive).
- 3..*: Three or more.

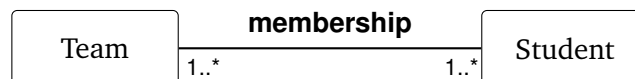
Role Names

We can indicate the behavior of an object in an association (i.e., the role of an object) using role names.



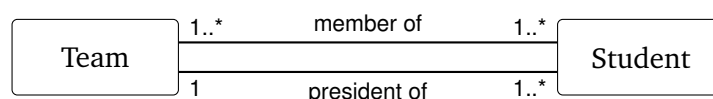
Association Names

We can also name the association itself to clarify the relationship.



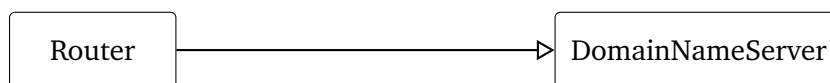
Dual Associations

Classes can have multiple associations between them with different roles.



Navigability

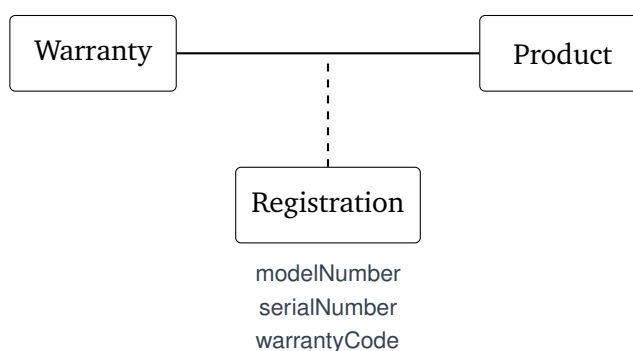
We can constrain the association relationship by defining the navigability. An arrow indicates the direction of visibility.



Here, a Router object requests services from a DNS object. The direction indicates that the server has no knowledge of the Router.

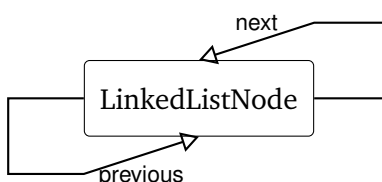
Association Classes

Associations can also be objects themselves, called link classes or association classes.



Self Association

A class can have a self association.

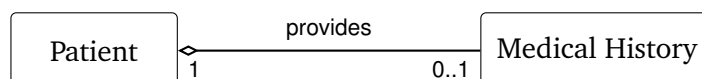


3. Aggregation and Composition

Both are special forms of association representing a **whole-part** relationship.

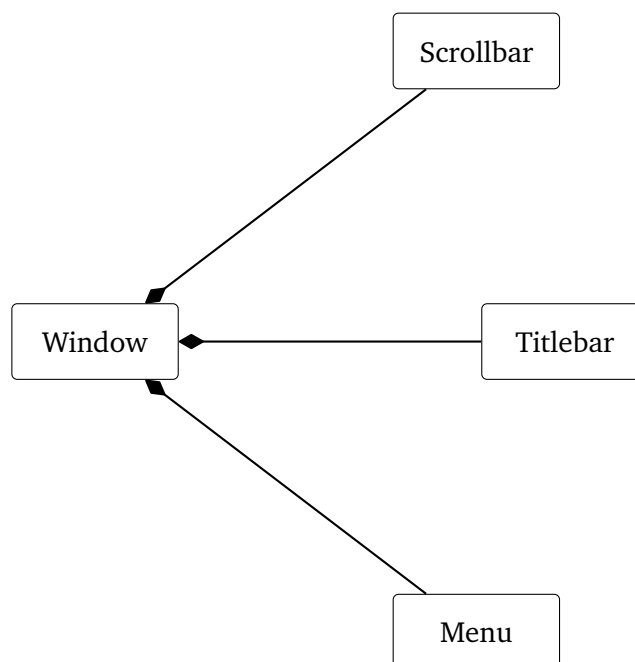
Aggregation (Shared Association)

We can model objects that contain other objects by way of special associations called aggregations and compositions. An aggregation specifies a whole-part relationship between an aggregate (a whole) and a constituent part, where the part can exist independently from the aggregate. Aggregations are denoted by a hollow-diamond adornment on the association.

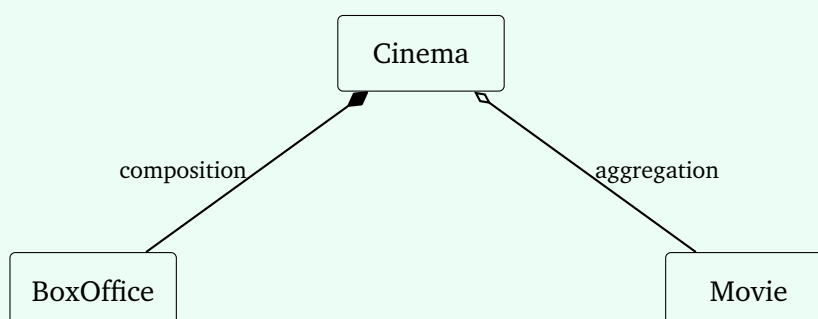


Composition (Composite Aggregation)

A composition indicates a strong ownership and coincident lifetime of parts by the whole (i.e., they live and die as a whole). Compositions are denoted by a filled-diamond adornment on the association.



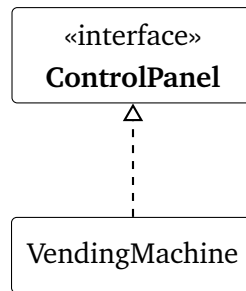
Comparison: Cinema



If the **Cinema** closes, the **Box Office** ceases to exist (Composition). However, the **Movies** still exist independently (Aggregation).

4. Interfaces

An **interface** is a named set of operations that specifies the behavior of objects without showing their inner structure. It can be rendered in the model by a one- or two-compartment rectangle, with the stereotype «interface» above the interface name.



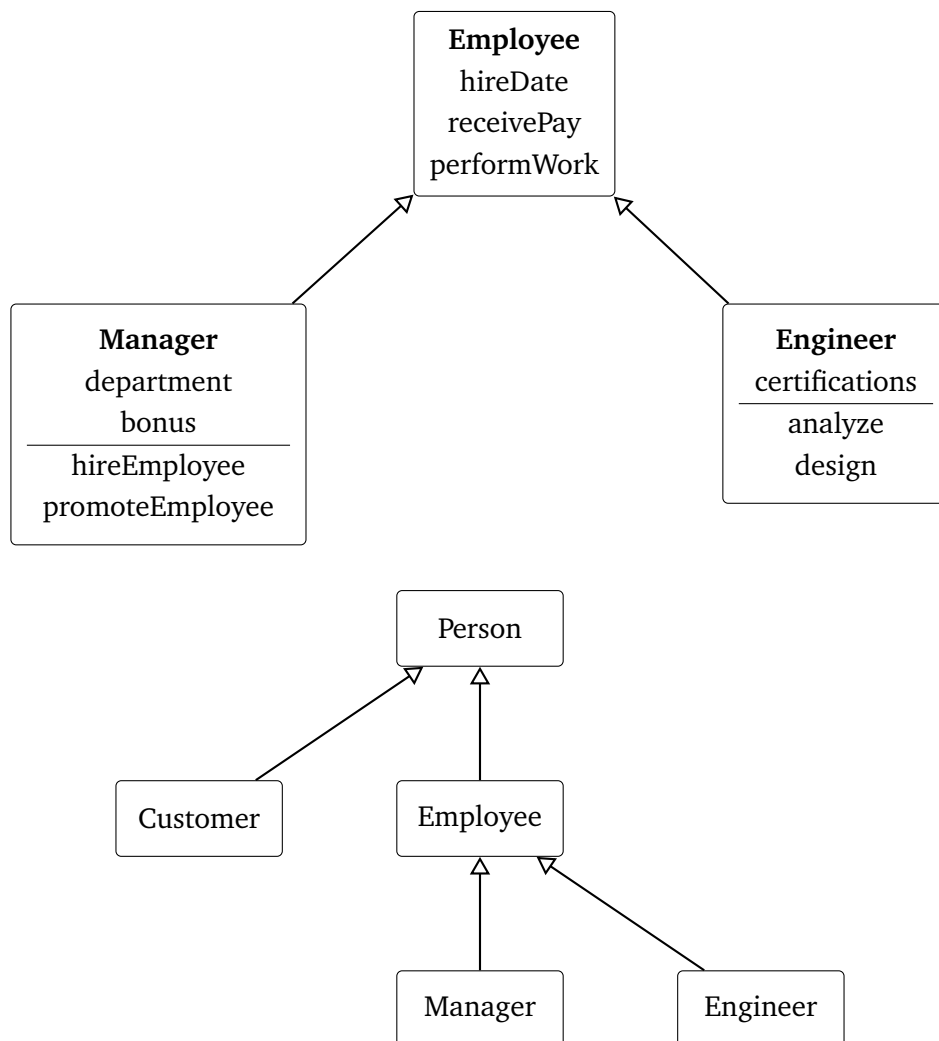
5. Generalization

Generalization

A specialized/generalized relationship in which objects of the specialized element (child) are substitutable for objects of the generalized element (parent). This is **inheritance**.

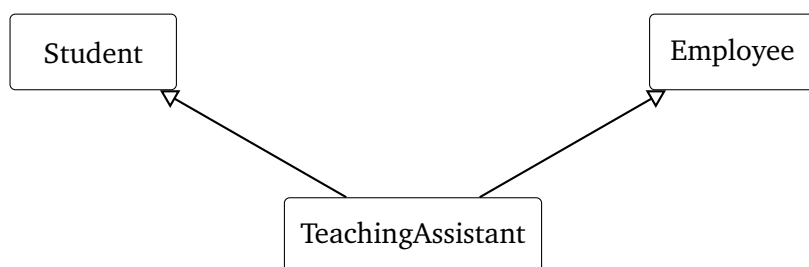
Notation

A solid line with a **hollow triangle** (arrowhead) pointing to the parent.



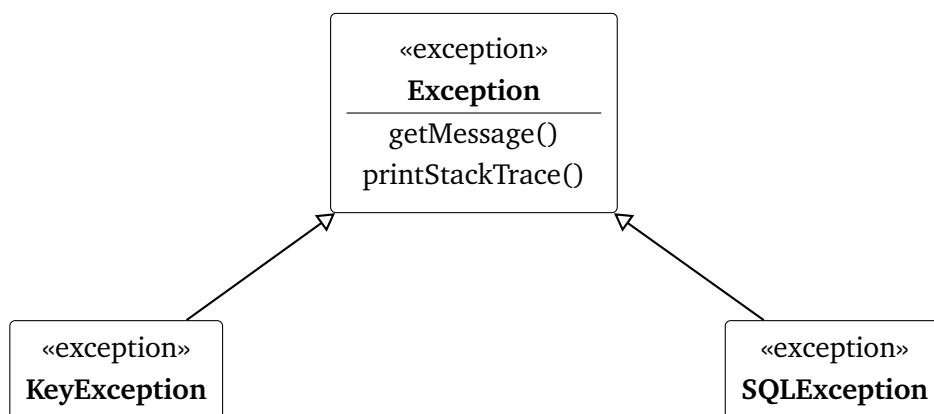
Multiple Inheritance

UML permits a class to inherit from multiple superclasses, although some programming languages (e.g., Java) do not permit multiple inheritance.



6. Exceptions

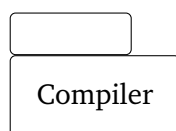
Exceptions can be modeled just like any other class. Notice the «exception» stereotype in the name compartment.



7. Packages

A **package** is a container-like element for organizing other elements into groups. A package can contain classes, other packages, and diagrams.

Basic Package Structure



Package Access Control

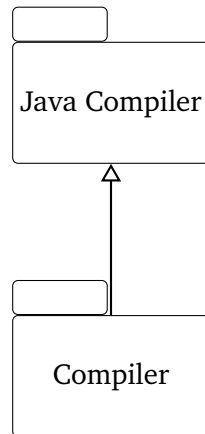
Packages can be used to provide controlled access between classes.



Here, classes in the BackEnd package have access to classes in the FrontEnd package.

Generalization between Packages

We can also model generalizations between packages.



8. Comprehensive Example: Video Store

A complete system often involves multiple relationship types working together.

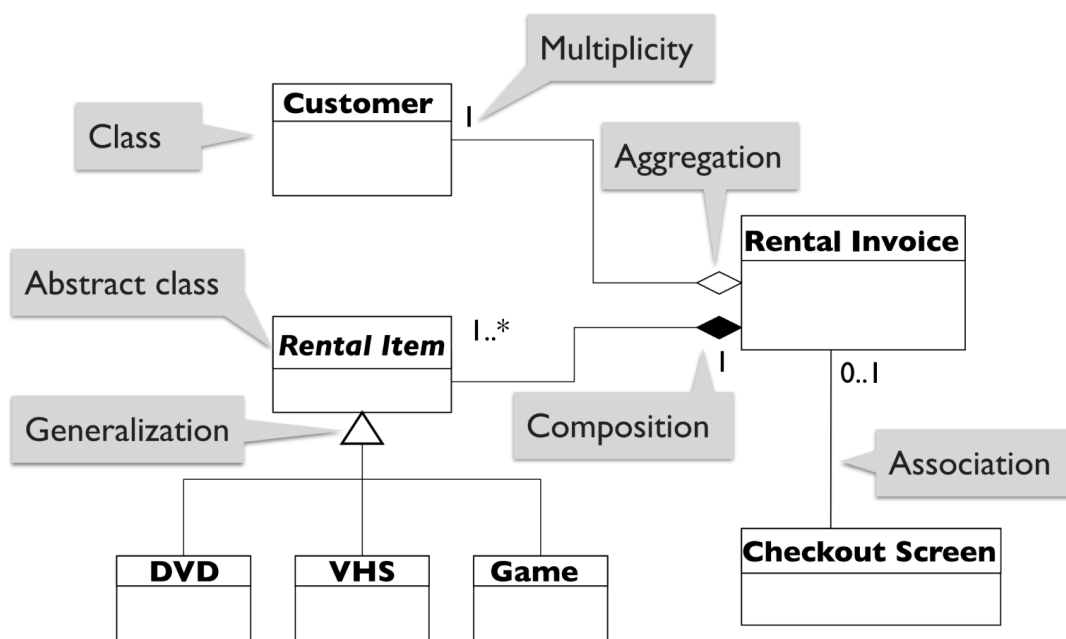


Figure 1: Comprehensive Video Store Diagram

Summary of Notations

Relationship	Notation	Meaning
Dependency	-----▷	Uses (Weak)
Association	————	Structural (Peer)
Aggregation	◊——	Has-a (Shared)
Composition	◆——	Contains-a (Exclusive)
Generalization	——▷	Is-a (Inheritance)